FIAP BANK | CHECKPOINT 5

Domain Driven Design - Java

Matheus Bispo Faria Barbosa - RM562140
Entrega individual | 2ESPH | Professor Eduardo dos Santos Ramos

1. Isolamento do domínio e infraestrutura relacional

No estado inicial, a Plataforma de Crédito concentrava domínio, persistência, serviços e controllers em um único módulo Maven. A classe model.SolicitacaoCredito representava a proposta de negócio e, ao mesmo tempo, a tabela relacional: importava jakarta.persistence e utilizava @Entity, @Table, @Id, @GeneratedValue e @Column. Assim, decisões de armazenamento contaminavam o modelo de negócio.

SolicitacaoCreditoRepository estendia diretamente JpaRepository e operava sobre essa mesma entidade. O serviço de concessão dependia dessa interface tecnológica, sem um contrato de repositório puro nem um adaptador que separasse domínio e persistência. Mudanças no ORM ou no banco poderiam afetar o modelo utilizado pelos casos de uso.

Essa conclusão foi conferida nas classes do commit inicial 3515708. O Core Bancário ainda não existia nesse estado inicial e foi construído durante o trabalho. Angular e Mesa são clientes da Plataforma: seus DTOs e clientes HTTP não são, por si, violações do domínio bancário.

2. Rotas que violavam os princípios RESTful

CreditoOperacoesNonRestfulController publicava todos os casos de uso por POST sob /api/credito. As consultas /consultarStatusCredito, /buscarSolicitacaoPorId e /listarTodasSolicitacoes eram operações de leitura representadas como comandos POST. Isso não expressava a semântica de leitura segura do GET e dificultava o aproveitamento das convenções de cache e intermediários HTTP.

As URIs /solicitarCredito, /aprovarSolicitacao e /reprovarSolicitacao descreviam ações, caracterizando um estilo RPC. A criação pode ser representada por POST /api/v1/creditos, com 201 Created; a coleção e seus itens, por GET /api/v1/creditos e GET /api/v1/creditos/{id}; a decisão, pela atualização do sub-recurso PUT /api/v1/creditos/{id}/status.

No Core, os recursos são /api/v1/clientes e /api/v1/contas, com GET, POST, PUT e DELETE e os códigos de resposta previstos. POST de criação não é automaticamente idempotente. A operação incremental de depósito/saque especificada pelo enunciado usa PUT /contas/{numeroConta}/saldo, mas repetir esse comando altera novamente o saldo; uma evolução rigorosa exigiria chave de idempotência ou modelagem explícita de transações.

3. Estratégia de refatoração sem quebrar os clientes

Core e Plataforma foram separados nos módulos domain, infrastructure, application e presentation. O domínio permanece sem Spring ou JPA; reúne entidades, objetos de valor e contratos de repositório. A infraestrutura contém entidades JPA separadas, interfaces Spring Data e adapters que convertem os modelos. A aplicação orquestra os casos de uso e publica DTOs imutáveis em Java Records. A apresentação expõe HTTP e aplica @Valid aos contratos de entrada.

A hierarquia é presentation -> application; application -> infrastructure e domain; infrastructure -> domain. Regras Maven Enforcer impedem dependências diretas proibidas, e uma verificação estática na fase validate rejeita referências de código que atravessam as fronteiras. O Core possui objetos de valor Cpf, Email e Saldo, enquanto a Plataforma mantém seu motor de concessão no domínio puro. Lombok simplifica as entidades de infraestrutura sem adicionar dependências ao domínio.

O controller REST oferece os recursos novos. Em paralelo, o controller adaptador mantém os caminhos POST, nomes dos campos e formatos de resposta usados pelos frontends legados. Ambos delegam aos mesmos serviços; não há uma segunda implementação das regras de concessão. Assim, Angular e Thymeleaf continuam funcionando enquanto a API de recursos é disponibilizada.

Bean Validation e tratamento global de exceções padronizam falhas. A integração Feign preserva o contrato de conta e distingue ausência de conta (404 ou lista vazia) de indisponibilidade. Os testes cobrem CRUD e movimentação do Core, consulta Feign, validação das rotas legadas, decisão de crédito, renderização Thymeleaf e fluxos Angular. A execução foi validada com Java 21 e Spring Boot 4.1.1.